Deep Learning – 12

CNN

Topics to be covered

- Data Types to be passed into the CNN
- Depth wise separable CNN

Data Types

- The data used with a convolutional network usually consist of several channels, each channel being the observation of a different quantity at some point in space or time.

Valid Convolution vs Invalid convolution

- Convolution **only makes sense** when the **same kind of thing** is repeated along the dimension we're sliding over.
- If the dimension's variability comes from having **more or fewer observations of the same type**, convolution works well.
- If the dimension's variability comes from **different kinds of features being present or absent**, convolution is **not appropriate**.

- **When convolution makes sense**
- **Example: Variable-length audio recordings**
- Suppose we're doing speech recognition.
- Input: A waveform over time.
- Each timestep's feature = amplitude (or a spectrogram column, etc.).
- A recording could be **short** (2 seconds) or **long** (10 seconds), but every timestep is still *"sound at time t"*.
- Using a convolution filter over the time axis is valid because:
  - The filter can detect patterns (e.g., a "sh" sound) no matter where it occurs in time.
  - The nature of the data is the same everywhere in that dimension.

- **Image example (invalid convolution axis)**
- Imagine a “wide” image where:
- Left half = a photograph (pixels with RGB values)
- Right half = a **text label block** describing the image (“Cat, sitting, outdoor”)
- If we convolve across the horizontal axis without distinction:
- A filter trained to detect vertical edges in the photo half will slide into the text block, which is a totally different representation (text as pixels, often high-contrast).
- The filter's learned pattern (edges in natural images) doesn’t apply to the text half — again, breaking the “same type everywhere” rule.

Different data types on which Convolution is done

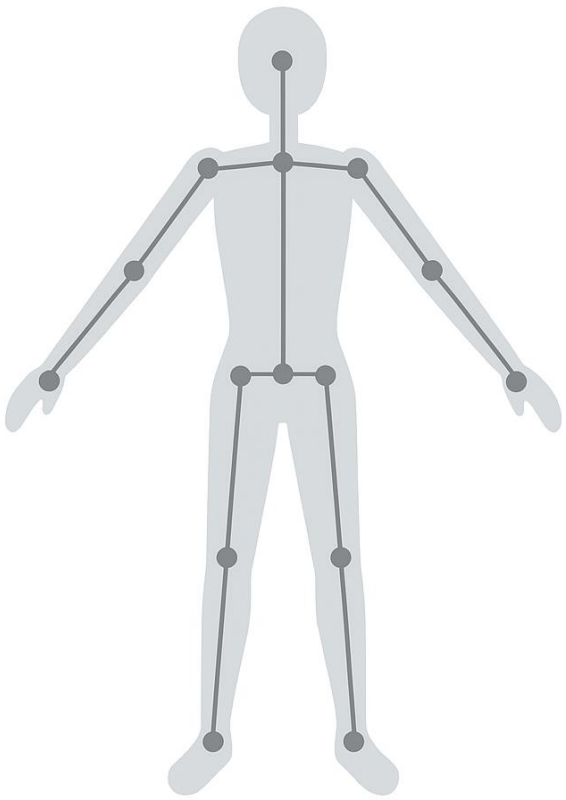
- 1D Convolution for Audio Waveforms
- **Convolution process**
- A 1D convolutional filter slides along time, looking at a **small window** of samples at a time, and computing a weighted sum.
- This helps the network learn patterns like:
  - short bursts (like a drum hit)
  - repeating cycles (like a note in music)
  - transitions (like speech phonemes)

Time step	Amplitude
t=0	0.2
t=1	0.4
t=2	0.5
t=3	0.3
t=4	0.1

- If we have a kernel/filter of size 3:
- $[w_0, w_1, w_2] = [0.5, 0.25, -0.5]$
- (centered on $t=1$):
- $(0.2)(0.5)+(0.4)(0.25)+(0.5)(-0.5)=0.1+0.1-0.25=-0.05$
- We then slide the kernel along **time** and repeat.

1D – Multichannel

- Skeleton animation data structure
- A 3D character's movement is driven by a **skeleton** — a hierarchy of joints (hips, knees, elbows, shoulders, etc.).
- At any given **time step**, the character's pose can be described by **joint angles** (e.g., hip rotation = 20°, knee bend = 45°).
- These angles are typically expressed around **three axes** (x, y, z) for each joint → that's **3 channels per joint**.
- So ,if you have:
- 15 joints
- Each with 3 rotation axes (channels)
- You get:
- 15×3=45



Joint	X Rotation	Y Rotation	Z Rotation
Head	15°	10°	-5°
Neck	10°	39°	0°
Left shoulder	15°	26°	2°
Left elbow	15°	31°	5°
Left wrist	20°	40°	0°
Right shoulder	20°	18°	9°
Right elbow	25°	16°	6°
Spine	20°	40°	10°
Left hip	10°	2°	5°
Left knee	10°	10°	10°
Right ankle	40°	10°	5°

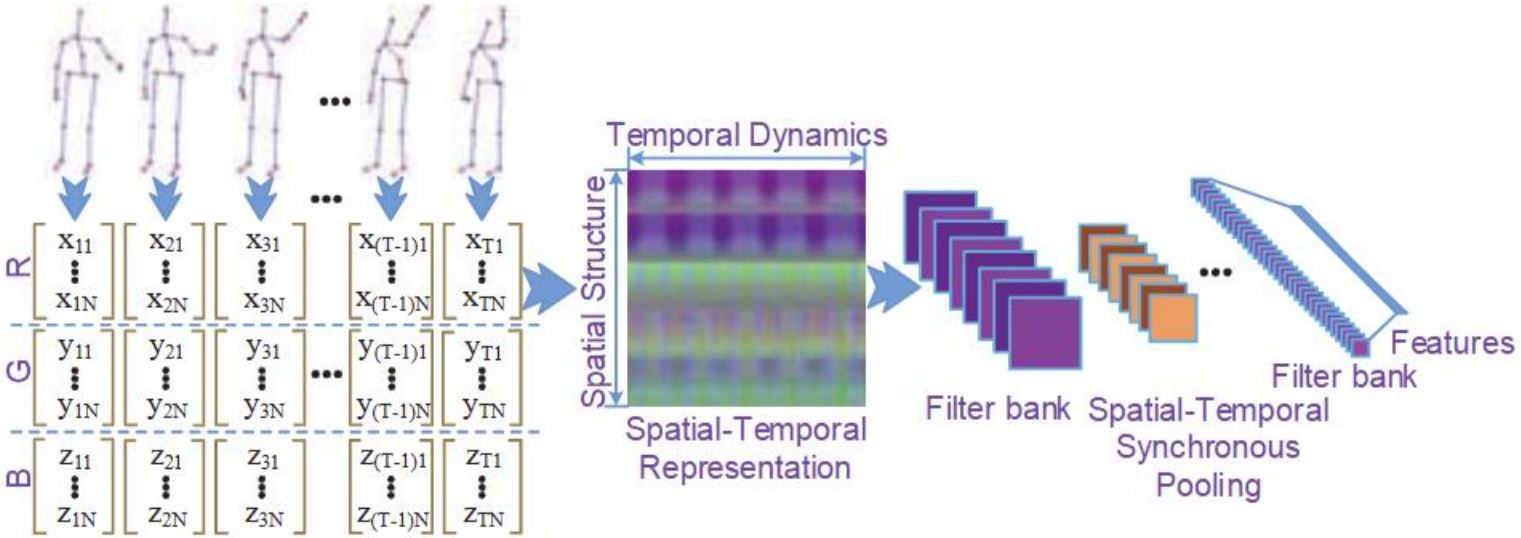
15 × 3 = 45 channels

- **Convolution process**
- A **1D convolutional kernel** moves along the **time dimension**.
- At each time step, the filter “looks” at a short **window** (e.g., 5 frames) and **all channels simultaneously**.
- This allows the model to learn temporal motion patterns:
  - Coordinated arm and leg movement in walking.
  - Synchronized shoulder rotation when swinging a bat.
  - Timing of a character’s jump.

Time	Hip-X	Hip-Y	Hip-Z	Knee-X	Knee-Y	Knee-Z
t=0	10°	0°	0°	5°	0°	0°
t=1	15°	0°	0°	10°	0°	0°
t=2	20°	0°	0°	15°	0°	0°
t=3	25°	0°	0°	20°	0°	0°

- A kernel of size 2 will look at **two consecutive time steps** and **all channels at once** to detect motion trends — e.g., “hip-X increasing + knee-X increasing” means stepping forward.

Skeleton Based Action Recognition with Convolutional Neural Network



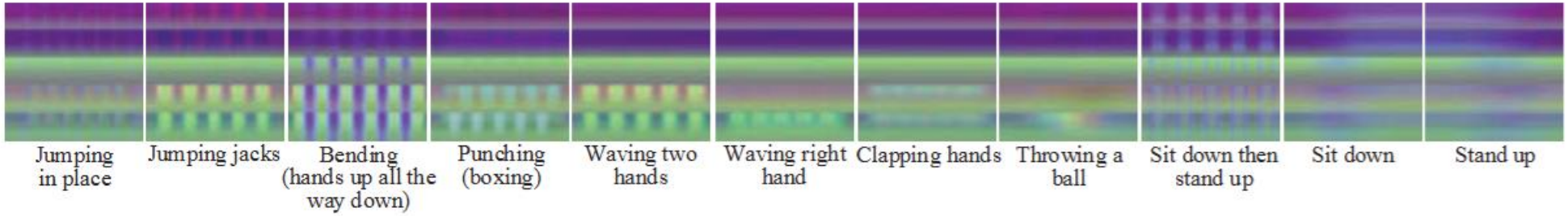


Figure 2: Image representations obtained on the Berkeley MHAD dataset [12].

- Instead of using all joints in a random order, we follow **human anatomy**:
  - Left arm
  - Right arm
  - Left leg
  - Right leg
  - Trunk (head, neck, spine, hips)

- Step 2 — Arrange joints in each part by physical connection
  - Example for left arm:[hand → wrist → elbow → shoulder]

- Do this for **all 5 parts**, then **concatenate** the 5 parts into a single vector for that frame.
- So, if each joint has coordinates (x, y, z), and there are M joints total after ordering:
- Frame 1 vector: [x1, x2, ..., xM]
- Frame 2 vector: [x1, x2, ..., xM]
- ...

- **Use three orthogonal projections as RGB**
- For each frame i:
- **R channel (Ri)** = all **x** coordinates of ordered joints
- **G channel (Gi)** = all **y** coordinates
- **B channel (Bi)** = all **z** coordinates
- So, frame i → one column in an RGB image:

- **Step 4 — Stack frames in chronological order**
- Each frame vector (for R, G, B) becomes a **column** in the image.
- Time flows **horizontally** across the image.
- Vertical axis = joint order (grouped by body parts).
- This way:
  - Vertical structure = spatial relationships.
  - Horizontal direction = temporal motion.

- The resulting RGB values are originally **float coordinates** (possibly negative, large, small).
- To store them as an image:
  - Normalize them to a fixed range (e.g., 0–255).
  - Convert to integers (so each pixel value is an integer in image format).
- Now we have a **color image** where:
- Vertical axis = skeleton structure.
- Horizontal axis = time steps.
- RGB color at each pixel encodes the 3D joint position.

2D Audio

- Raw audio is just **amplitude over time** (1D).
  If we apply a **Fourier transform** (e.g., Short-Time Fourier Transform, STFT), we get:
- **Horizontal axis (columns)** = time steps (chunks of the original audio)
- **Vertical axis (rows)** = frequency bins (low → high)
- **Value in each cell** = magnitude of that frequency at that time

- A **2D convolution kernel** can move in two directions:
- **Time axis** → detects repeating temporal patterns
  (e.g., rhythm, beats, speech phonemes)
- **Frequency axis** → detects harmonic patterns across frequencies
  (e.g., chords, timbre, tone)

- **Equivariance meaning in this context**
- **Convolution in time** = **time-shift equivariance**
  → If you play the same melody later in time, the features in the CNN will shift horizontally but remain the same pattern.
- **Convolution in frequency** = **frequency-shift equivariance**
  → If the same melody is played in a different octave (higher/lower pitch), its spectrogram shifts **vertically**, and the CNN still detects it, just at a different vertical position.

2D – Image

- Color image data: One channel contains the red pixels, one the green pixels, and one the blue pixels. The convolution kernel moves over both the horizontal and the vertical axes of the image, conferring translation equivariance in both directions

3D - Datatype

- 3-D
- Volumetric data: A common-source of this kind of data is medical imaging technology, such as CT scans.

- Instead of just a 2D image (height × width), volumetric data adds a third spatial dimension: depth. For CT (Computed Tomography) scans, this depth dimension corresponds to slices through the body. Imagine taking a loaf of bread and slicing it into thin layers — each slice is like one CT image. Stacking them back together gives you a 3D volume.

- Height × Width × Depth × Channels
- Imagine detecting a small ball inside a loaf of bread:
- **2D convolution** would see just one circular spot per slice — no context that it’s part of a 3D sphere.
- **3D convolution** can detect the entire spherical shape across slices.
- CT images are black and white.

3D Multichannel

- Color video data: One axis corresponds to time, one to the height
- of the video frame, and one to the width of the video frame.
  - Time × Height × Width × Channels

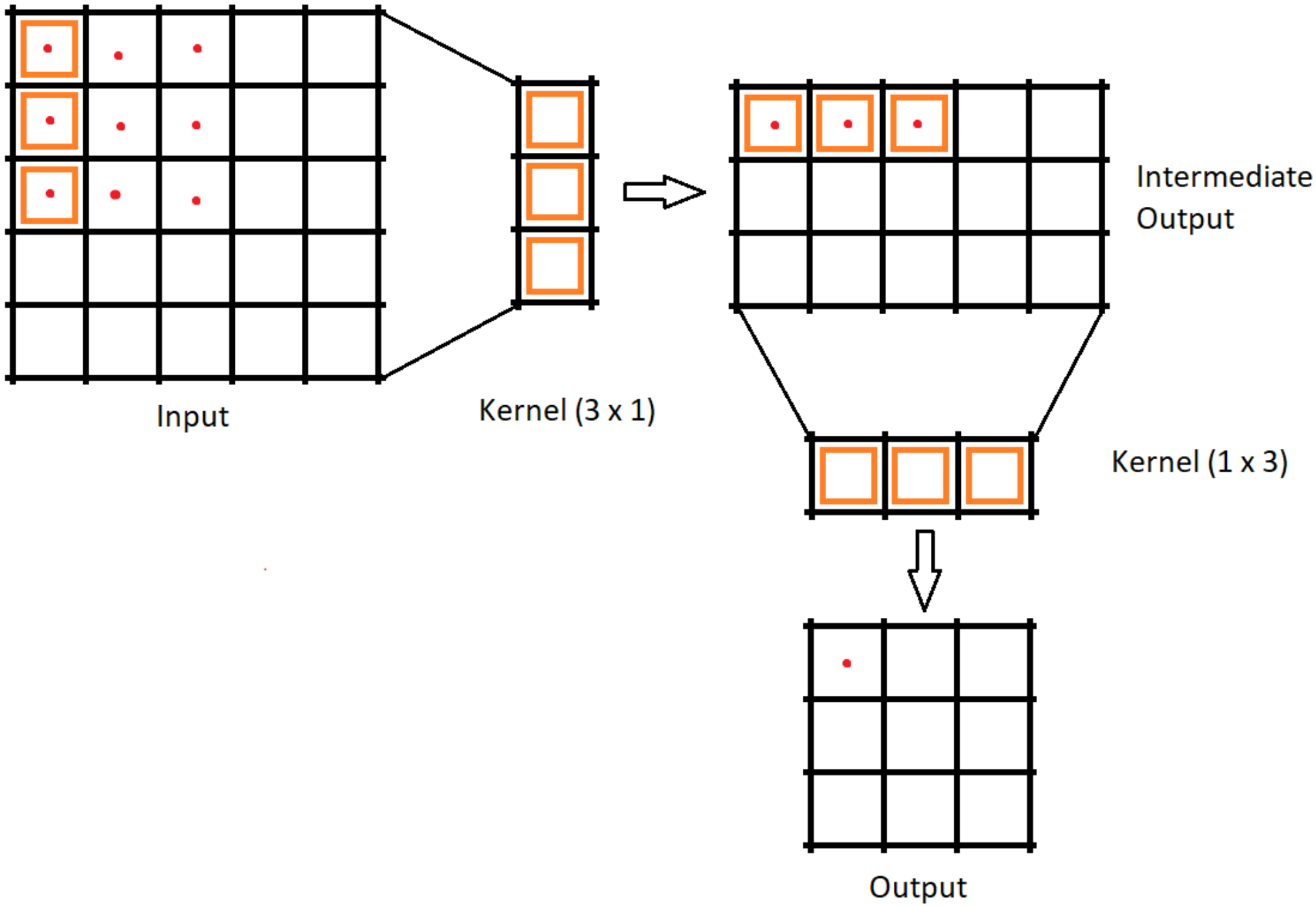
Efficient Convolution Algorithms

- When a d-dimensional kernel can be expressed as the outer product of d vectors, one vector per dimension, the kernel is called separable.

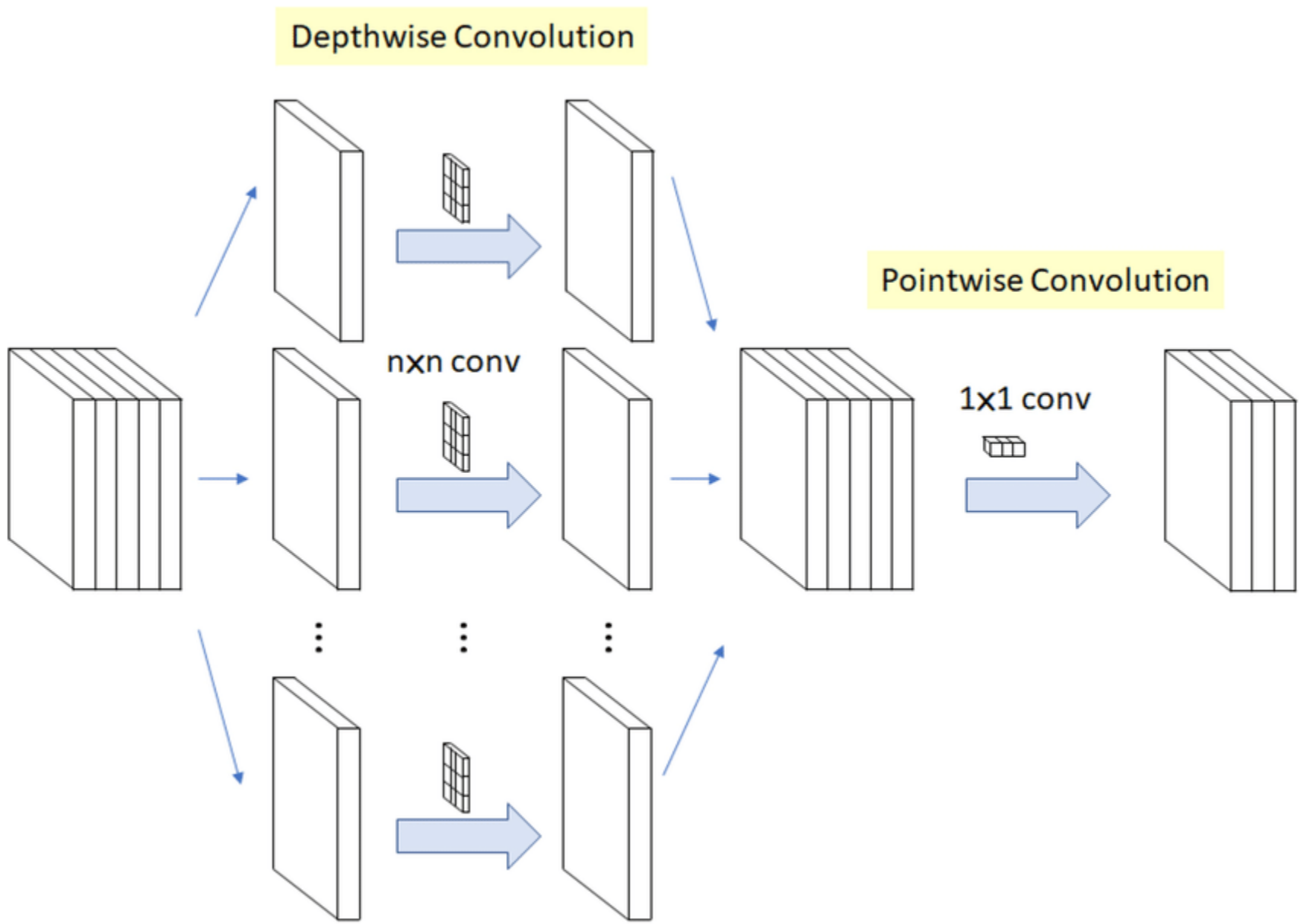
In images height and width are called spatial axes. The kernel that can be separated across spatial axes is called the spatially separable kernel. The kernel is broken into two smaller kernels and those kernels are multiplied sequentially with the input image to get the same effect of the full kernel.

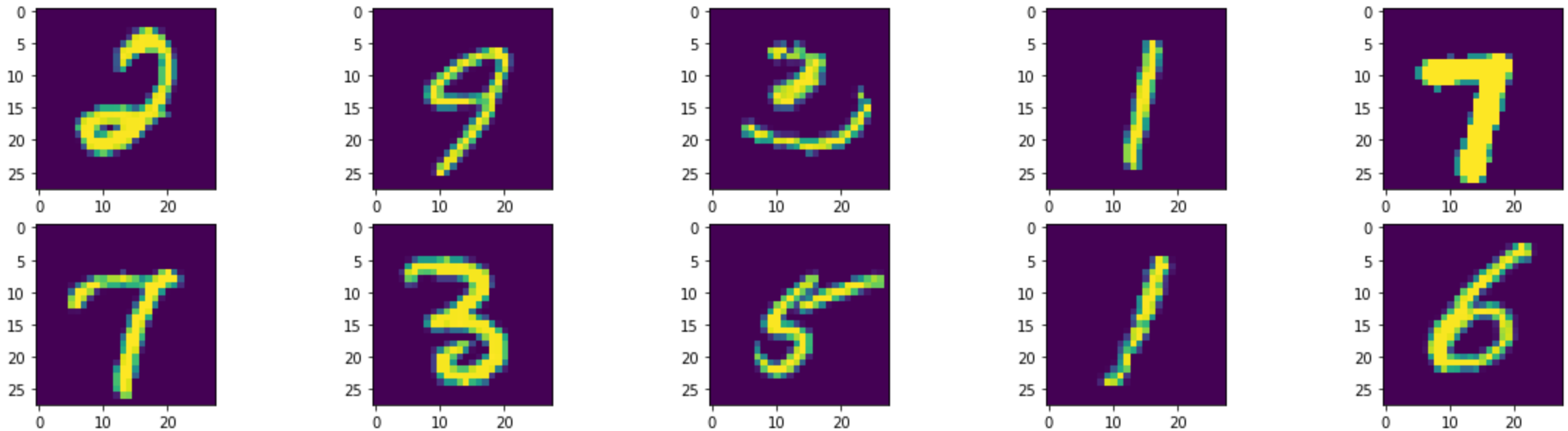
For example

$$\begin{bmatrix} +1 & 0 & -1 \\ +1 & 0 & -1 \\ +1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$$


Input
Kernel (3 x 1)
Intermediate
Output
Kernel (1 x 3)
Output

- In the conventional convolutions, if the kernel is 3 x 3 then the number of parameters would be 9. In spatially separable convolution we divide the kernel into two kernels of shapes 3 x 1 and 1 x 3. The input is first convolved with 3 x 1 kernel and then with 1 x 3, then the number of parameters would be 3 + 3 = 6.


Depthwise Convolution
nxn conv
Pointwise Convolution
1x1 conv


0
5
10
15
20
25
0
10
20

Model with Conventional CNNs

```
model = tf.keras.Sequential([
        tf.keras.layers.Conv2D(8, 7, input_shape=(28,28,1), strides=2),
        tf.keras.layers.Conv2D(16, 5, strides=2),
        tf.keras.layers.Conv2D(32, 3),
        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(loss='categorical_crossentropy', metrics=['acc'])
model.summary()
```

```
Model: "sequential_17"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_43 (Conv2D)          (None, 11, 11, 8)         400

 conv2d_44 (Conv2D)          (None, 4, 4, 16)          3216

 conv2d_45 (Conv2D)          (None, 2, 2, 32)          4640

 global_average_pooling2d_17  (None, 32)               0
  (GlobalAveragePooling2D)

 dense_17 (Dense)            (None, 10)                330

=================================================================
Total params: 8,586
Trainable params: 8,586
Non-trainable params: 0
```

```
model = tf.keras.Sequential([
        tf.keras.layers.SeparableConv2D(8, 7, input_shape=(28,28,1), strides=2),
        tf.keras.layers.SeparableConv2D(16, 5, strides=2),
        tf.keras.layers.SeparableConv2D(32, 3),
        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(loss='categorical_crossentropy', metrics=['acc'])
model.summary()
```

```
Model: "sequential_18"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 separable_conv2d_9 (Separab  (None, 11, 11, 8)        65
 leConv2D)

 separable_conv2d_10 (Separa  (None, 4, 4, 16)         344
 bleConv2D)

 separable_conv2d_11 (Separa  (None, 2, 2, 32)         688
 bleConv2D)

 global_average_pooling2d_18  (None, 32)               0
  (GlobalAveragePooling2D)

 dense_18 (Dense)            (None, 10)                330

=================================================================
Total params: 1,427
Trainable params: 1,427
Non-trainable params: 0
_________________________________________________________________
```

- Input channels here = **8** (from previous layer output).
- **Depthwise params**: $5 \times 5 \times 8 = 200$
- **Pointwise params**: $8 \times 16 = 128$
- **Bias**: 16
- **Total**: $200 + 128 + 16 = 344$

Layer-by-layer parameter counts (spatially separable row×col)

1. **Layer 1** — $k = 7,\ C_{in} = 1,\ C_{out} = 8$

$$\text{params} = 2 \cdot 7 \cdot 1 \cdot 8 + 8 = 112 + 8 = \boxed{120}$$

2. **Layer 2** — $k = 5,\ C_{in} = 8,\ C_{out} = 16$

$$\text{params} = 2 \cdot 5 \cdot 8 \cdot 16 + 16 = 1280 + 16 = \boxed{1296}$$

3. **Layer 3** — $k = 3,\ C_{in} = 16,\ C_{out} = 32$

$$\text{params} = 2 \cdot 3 \cdot 16 \cdot 32 + 32 = 3072 + 32 = \boxed{3104}$$

4. **GlobalAveragePooling2D** — no parameters: $\boxed{0}$
5. **Dense (32 → 10)**

$$\text{params} = 32 \cdot 10 + 10 = 320 + 10 = \boxed{330}$$

Basic strategies for obtaining convolution kernels without supervised training.

- Simply initialize randomly: random filters work well in convolutional networks. Inexpensive way to choose the architecture of a convolutional network:
  - Evaluate the performance of several convolutional network architecture by training only the last layer
  - Take the best of these architectures and train the entire architecture using a more expensive approach.

- Suppose you want to compare two architectures on CIFAR-10:
- **Architecture A**: 3 conv layers, 32 filters each
- **Architecture B**: 5 conv layers, 64 filters each
- 👉 Training both fully on CIFAR-10 might take hours.
- Instead:
- Initialize all convolution filters randomly (freeze them).
- Run data through the network, extract features from the last convolutional output.
- Train only the final softmax classifier.
  - This might take minutes, not hours.
- Compare validation accuracies.
  - If A gets ~25% accuracy and B gets ~40% accuracy with random filters, B is probably the better architecture.
- Now, fully train only B (the better candidate) with gradient descent on all layers.

- 2. Design them by hand

•Here, one manually specifies filters, such as edge detectors (Sobel, Gabor filters) or filters that capture certain orientations or scales.
•This method is less common today but was historically important.
•This is inspired by early computer vision and neuroscience.

- 3. Learn the kernel with an unsupervised criterion:
  - Learning the features from unsupervised criterion allows them to be determined separately from the classifier layer at the top of the architecture.

Framework for Unsupervised Feature Learning

- 1. Extract random patches
- Suppose each training image is $n \times n$ with d channels (say RGB → d=3).Pick random sub-patches of size $w \times w \times d$.
- Each patch is flattened into a vector:$x(i) \in R^N$,$N = w \cdot w \cdot d$
- Dataset of m patches:$X = \{x(1), x(2), \ldots, x(m)\}$,$x(i) \in R^N$
- ✅ Example: CIFAR-10 images are 32×32×3. If w=6, then each patch is 6×6×3=108-dimensional.

- 2. Pre-processing

Each patch is normalized to remove brightness and contrast effects.

- Normalize:

$$\tilde{x}^{(i)} = \frac{x^{(i)} - \mu(x^{(i)})}{\sigma(x^{(i)})}$$

where $\mu(x^{(i)})$ is the mean pixel value and $\sigma(x^{(i)})$ its standard deviation.

- (Optional) Whitening:

  Make features less correlated:

$$\tilde{X} = U\Lambda^{-\frac{1}{2}}U^{\top}X$$

where X is the data covariance, U are eigenvectors, and Λ eigenvalues.

This makes the covariance matrix of patches approximately the identity matrix.

Fig.1: 10 raw sample images from each of 10 different classes of CIFAR10 namely airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

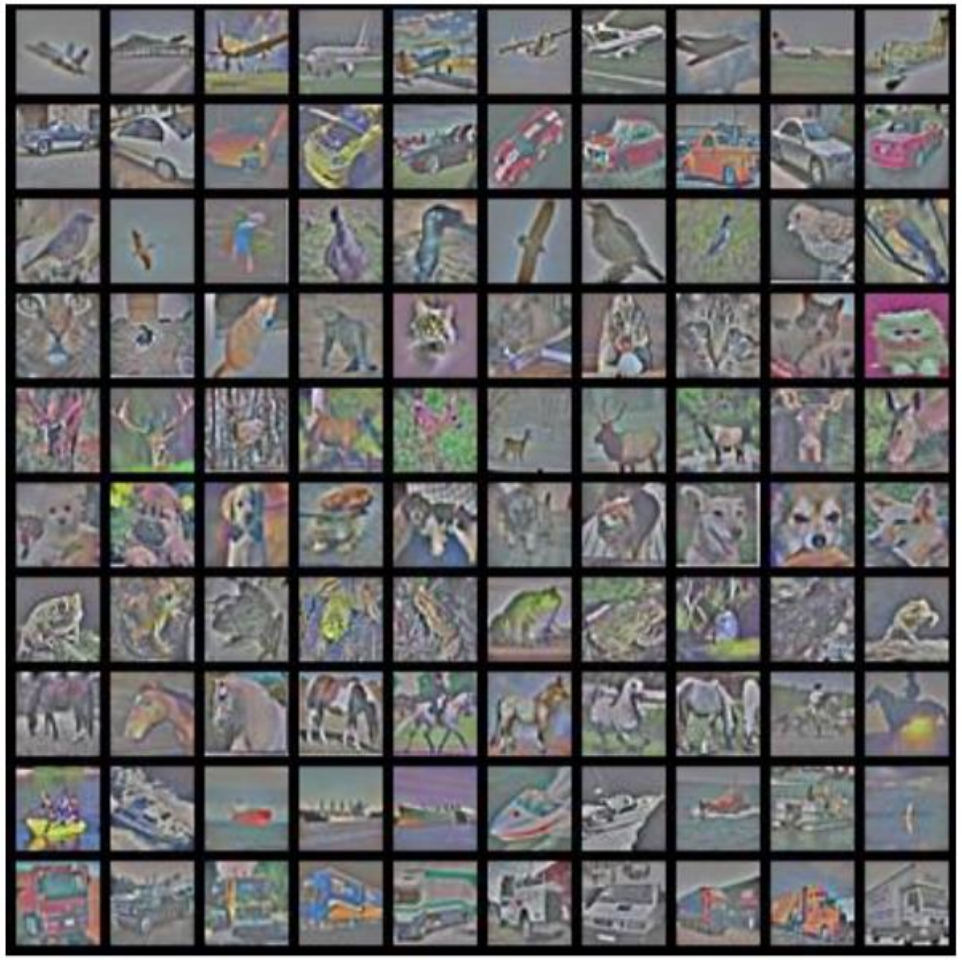


Fig.2:ZCA-whitened images of sample images of Fig. 1 with ε=0.1

A. *Mean Normalization*

The mean along each of the features (dimensions of images) of training samples is computed and subtracted from each of the images. This way the whole training set is turned into organized data. Thus the brightness of the whole training set is normalized with respect to each dimension. This is done by(1).

$$X' = X - \mu. \qquad (1)$$

where X' is the normalized data, X is the original data and μ is the mean vector across all the features of X.

B. *Standardization*

The data is first mean normalized and then the standard deviation along each of the features of training samples is computed and divided. This makes the final data mean and variance normalized. Thus the raw data is organized with respect to both mean and variance of each of the dimension of the training set. This is standardization of input data X done by (2).

$$X' = (X - \mu) / \Sigma. \qquad (2)$$

where X' is the normalized data, μ is the mean vector across all the features and Σ is the standard deviation vector across all the features

ZCA

The method of performing ZCA is mentioned in details. First the data(X) is size normalized by feature scaling with (3).

$$X' = X / 255. \qquad (3)$$

Then it is mean normalized by (1) and later the singular value decomposition of the covariance matrix of the mean normalized data is calculated. Finally, the whitening is performed by (4).

$$X_{ZCA} = U.\mathrm{diag}(1 / \sqrt{\mathrm{diag}(S) + \varepsilon}).U^T.X'. \qquad (4)$$

where diag(a) represents the diagonal matrix of given matrix a, U is the Eigen vector matrix and S is the Eigen value matrix of singular value decomposition of covariance matrix, U^T is the transpose of the Eigen vector matrix U, ε is the whitening coefficient. This method of preprocessing introduces another hyper-parameter whitening coefficient(ε).

3. K-means clustering

- Group patches into K clusters.
- Each cluster center (centroid) acts like a “filter.”
- Example: One cluster might represent vertical edges, another horizontal edges

Stage 4: Feature Extraction from Full Images

Once f is learned, we apply it convolutionally to images:

- Slide a window of size $w \times w$ across the input image with stride s.
- At each location (i, j), extract patch x_{ij}, compute $y_{ij} = f(x_{ij})$.
- Result: a feature map of size $(n - w + 1)/s \times (n - w + 1)/s \times K$.

✅ Example: For CIFAR-10 (32×32 image), $w = 6$, $s = 1$, $K = 100$ → output feature map is $27 \times 27 \times 100$.

- **Pooling**
- The feature map is still big.
- To shrink it, we use **pooling**.
- Example: Divide the image into 4 regions (quadrants).
- For each region, sum up (or average) the feature values.
- This gives us a **compact representation** of the image.

- **Classification**
- Now that we have compact feature vectors for each image, we can train a simple classifier (like a linear SVM or logistic regression).
- This classifier predicts the label (cat, dog, car, etc.) using the features.

Suppose we're working on CIFAR-10 (small color images).

1. Extract **random 6×6 patches** → flatten into 108 numbers.
2. Normalize patches.
3. Use **K-means** with $K = 100$ clusters → each cluster = a filter (edge, color blob, texture).
4. Slide these 100 filters over a full image → get feature maps.
5. Pool over 4 quadrants → final vector has $4 \times 100 = 400$ numbers.
6. Train an SVM classifier on these 400 numbers → predict the label.

✅ Even without labels during feature learning, this works surprisingly well (historically ~80% accuracy on CIFAR-10).

Video Classification using CNN

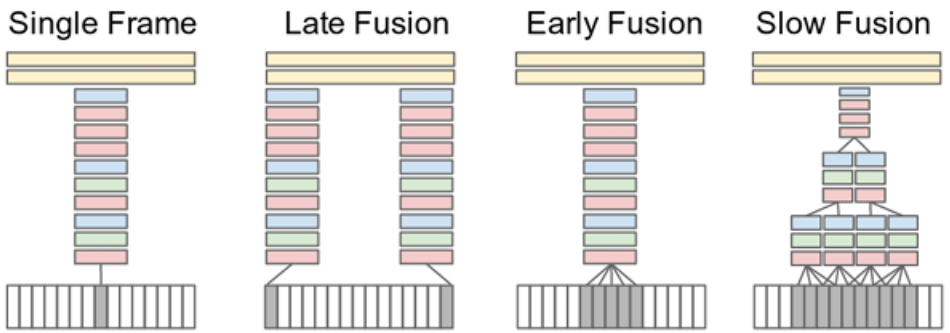


Figure 1: Explored approaches for fusing information over temporal dimension through the network. Red, green and blue boxes indicate convolutional, normalization and pooling layers respectively. In the Slow Fusion model, the depicted columns share parameters.

Reference

- https://medium.com/free-code-camp/https-medium-com-hadrienj-preprocessing-for-deep-learning-9e2b9c75165c